

PLAYTO PAYOUT ENGINE – FULL RUN GUIDE

This document explains what commands to run and what happens at each step.

1. CLONE PROJECT

Command:

```
git clone https://github.com/Zeeshan128/playto-payout-engine  
cd playto-payout-engine
```

What happens:

- Downloads project code from GitHub
- Moves into project directory

2. CREATE VIRTUAL ENVIRONMENT

Command:

```
python -m venv .venv  
.venv\Scripts\activate
```

What happens:

- Creates isolated Python environment
- Avoids conflicts with system packages

3. INSTALL DEPENDENCIES

Command:

```
pip install -r requirements.txt
```

What happens:

- Installs Django, DRF and required libraries

4. RUN MIGRATIONS

Command:

```
python manage.py migrate
```

What happens:

- Creates database tables (Merchant, LedgerEntry, Payout)
- Prepares database schema

5. SEED INITIAL DATA

Command:

```
python manage.py seed_data
```

What happens:

- Creates merchants
- Adds credit entries (initial funds)
- Balance becomes available for testing

6. START SERVER

Command:

```
python manage.py runserver
```

What happens:

- Starts Django backend server
- API available at <http://127.0.0.1:8000/>

7. RUN FRONTEND

Command:

cd frontend

npm install

npm start

What happens:

- Installs React dependencies
- Starts frontend at <http://localhost:3000>
- UI connects to backend APIs

8. ADD FUNDS

Command:

POST /api/v1/add-funds/1/

What happens:

- Creates credit ledger entry
- Increases merchant balance

9. CREATE PAYOUT

Command:

POST /api/v1/payouts/1/

What happens:

- Checks balance safely
- Deducts funds (debit entry)
- Creates payout record
- Starts background processing

10. PAYOUT PROCESSING

What happens internally:

- Status: pending → processing
- Random outcome:
- 70% → completed
- 20% → failed (refund happens)
- 10% → retry

11. REFUND LOGIC

If payout fails:

- Credit entry added
- Balance restored
- Done atomically with DB transaction

12. CONCURRENCY SAFETY

Uses:

`select_for_update()`

What happens:

- Locks merchant row
- Prevents double spending

13. IDEMPOTENCY

Same request with same key:

- Returns same payout
- No duplicates created

FINAL RESULT

System ensures:

- Accurate money handling
- No race conditions
- Safe retries
- Real-world fintech behavior